Being good programmers, we'll write a unit test for this. Be warned, though—it's messy (and we'll improve on it shortly):

ducktyping/test_add_customer_1.rb

```
require "minitest/autorun"
require_relative "add_customer"

class TestAddCustomer < Minitest::Test
  def test_add
    customer = Customer.new("Ima", "Customer")
    File.open("tmpfile", "w") do |f|
      customer.append_name_to_file(f)
    end
    File.open("tmpfile") do |f|
      assert_equal("Ima Customer", f.gets)
    end
  ensure
    File.delete("tmpfile") if File.exist?("tmpfile")
  end
end
```

produces:

```
Run options: --seed 3704
# Running:

.
Finished in 0.000836s, 1196.1722 runs/s, 1196.1722 assertions/s.
1 runs, 1 assertions, 0 failures, 0 errors, 0 skips
```

We have to do all that work in the test to create a file to write to, then reopen it, and read in the contents to verify the correct string was written. We also have to delete the file when we've finished (but only if it exists).

Because Ruby is dynamic, we don't actually have to do all that work on actual files to make the test run. Instead, we could rely on duck typing. All we need is something that walks like a file and quacks like a file that we can pass in to the method under test. And all that means *in this circumstance* is that we need an object that responds to the << method by appending something. Do we have something that does this? How about a humble String?

ducktyping/test_add_customer_2.rb

```
require "minitest/autorun"
require_relative "add_customer"

class TestAddCustomer < Minitest::Test
  def test_add
    customer = Customer.new("Ima", "Customer")
    fake_file = ""
    customer.append_name_to_file(fake_file)
    assert_equal("Ima Customer", fake_file)
  end
end
```

produces:

```
Run options: --seed 59163
# Running:

.
Finished in 0.000359s, 2785.5153 runs/s, 2785.5153 assertions/s.
```